

activity-01

August 25, 2026

1 Activity 01: Hacking Password

1.1 Import External Libraries

```
[19]: import time
import bcrypt
import hashlib
import itertools
import pandas as pd
import tqdm.notebook as tqdm
```

1.2 Example of hashing a password

```
[2]: # SHA1
m = hashlib.sha1(b"Chulalongkorn").hexdigest()
print(f"Hashed via SHA1:    {m}")

# MD5
m = hashlib.md5(b"Chulalongkorn").hexdigest()
print(f"Hashed via MD5:    {m}")

# BCRYPT
salt = bcrypt.gensalt()
m = bcrypt.hashpw(b"Chulalongkorn", salt).decode("utf-8")
print(f"Hashed via BCRYPT:   {m}")
```

Hashed via SHA1: ca8a68498ae67cd14c15f5ebf043633224005759

Hashed via MD5: 46fa3b56c660faff420190c18c98a56b

Hashed via BCRYPT: \$2b\$12\$0L1x1D1c8xx009.Vaa0al.3AbcJPzAGzwtEE/Br/pCyBD.wjdcvd2

1.3 Download the Password List

Download the password list from the following link: <https://github.com/danielmiessler/SecLists/blob/master/Passwords/10k-most-common.txt>

```
[3]: with open("10k-most-common.txt", "r") as f:
      passwords = f.read().splitlines()

passwords[:10]
```

```
[3]: ['password',
      '123456',
      '12345678',
      '1234',
      'qwerty',
      '12345',
      'dragon',
      'pussy',
      'baseball',
      'football']
```

1.3.1 Define the Character Substitutions Dictionary

```
[13]: char_substitutions = {
      'o': '0',
      'i': '1',
      'l': '1',
      'z': '2',
      'e': '3',
      'a': '4',
      's': '5',
      'b': '6',
      't': '7',
      'g': '9',
      'q': '9',
      }
```

```
[14]: def generate_variations(password):
      char_options = []
      for c in password:
          opts = {c.lower(), c.upper()}
          if c.lower() in char_substitutions:
              opts.add(char_substitutions[c.lower()])
          char_options.append(opts)
      for combo in itertools.product(*char_options):
          yield "".join(combo)
```

1.4 Task 1

```
[18]: TARGET_HASH = "d54cc1fe76f5186380a0939d2fc1723c44e8a5f7"

for password in tqdm.tqdm(passwords):
    for cand in generate_variations(password):
        if hashlib.sha1(cand.encode()).hexdigest() == TARGET_HASH:
            print(f"Found password: {cand}")
```

0%| | 0/10000 [00:00<?, ?it/s]

Found password: ThaiLanD

1.5 Task 2

```
[34]: def bench_fast(algo, data=b"Chulalongkorn", duration=2.0):
    count, end = 0, time.perf_counter() + duration

    def _bench_bcrypt(rounds=12, data=b"Chulalongkorn", n=25):
        t = time.perf_counter()
        for _ in range(n):
            bcrypt.hashpw(data, bcrypt.gensalt(rounds=rounds))
        return n / (time.perf_counter() - t)

    def _bench_scrypt(n=25, data=b"Chulalongkorn", r=8, p=1, dklen=64):
        t = time.perf_counter()
        for _ in range(n):
            hashlib.scrypt(data, salt=b"Chulalongkorn", n=2**14, r=r, p=p,
            ↪dklen=dklen)
        return n / (time.perf_counter() - t)

    if algo in ["md5", "sha1", "sha256", "sha512"]:
        while time.perf_counter() < end:
            hashlib.new(algo, data).hexdigest(); count += 1
    elif algo == "bcrypt":
        count = _bench_bcrypt(data=data, n=25)
    elif algo == "scrypt":
        count = _bench_scrypt(data=data, n=25)

    return count / duration
```

```
[35]: print(f"MD5:      {bench_fast('md5')}      :,.2f} hashes/sec")
print(f"SHA1:     {bench_fast('sha1')}     :,.2f} hashes/sec")
print(f"SHA256:    {bench_fast('sha256')}    :,.2f} hashes/sec")
print(f"SHA512:    {bench_fast('sha512')}    :,.2f} hashes/sec")
print(f"bcrypt:    {bench_fast('bcrypt')}    :,.2f} hashes/sec")
print(f"scrypt:    {bench_fast('scrypt')}    :,.2f} hashes/sec")
```

MD5: 1,312,314.00 hashes/sec
SHA1: 1,427,464.00 hashes/sec
SHA256: 1,382,026.50 hashes/sec
SHA512: 1,011,048.00 hashes/sec
bcrypt: 2.37 hashes/sec
scrypt: 12.70 hashes/sec

1.6 Task 3

```
[38]: CHARECTER_NUM = 94 # 26 + 26 + 10 + 32

def display_time(sec):
    if sec < 60:
        return f"{sec:,.2f} seconds"
    elif sec < 60 * 60:
        return f"{sec / 60:,.2f} minutes"
    elif sec < 60 * 60 * 24:
        return f"{sec / 60 / 60:,.2f} hours"
    elif sec < 60 * 60 * 24 * 365:
        return f"{sec / 60 / 60 / 24:,.2f} days"
    else:
        return f"{sec / 60 / 60 / 24 / 365:,.2f} years"

for algo in ["md5", "sha1", "sha256", "bcrypt"]:
    print("-----")
    print(f"Brute-force time for {algo}:")

    algo_time = bench_fast(algo)
    for l in range(1, 9):
        print(f"Length {l}: {display_time(CHARECTER_NUM**l / algo_time)}")
```

```
-----
Brute-force time for md5:
Length 1: 0.00 seconds
Length 2: 0.01 seconds
Length 3: 0.61 seconds
Length 4: 57.22 seconds
Length 5: 1.49 hours
Length 6: 5.85 days
Length 7: 1.51 years
Length 8: 141.67 years
-----
```

```
Brute-force time for sha1:
Length 1: 0.00 seconds
Length 2: 0.01 seconds
Length 3: 0.59 seconds
```

Length 4: 55.28 seconds
Length 5: 1.44 hours
Length 6: 5.65 days
Length 7: 1.46 years
Length 8: 136.85 years

Brute-force time for sha256:

Length 1: 0.00 seconds
Length 2: 0.01 seconds
Length 3: 0.58 seconds
Length 4: 54.54 seconds
Length 5: 1.42 hours
Length 6: 5.58 days
Length 7: 1.44 years
Length 8: 135.03 years

Brute-force time for bcrypt:

Length 1: 39.74 seconds
Length 2: 1.04 hours
Length 3: 4.06 days
Length 4: 1.05 years
Length 5: 98.38 years
Length 6: 9,247.61 years
Length 7: 869,275.22 years
Length 8: 81,711,870.76 years

If we set password length to 8 characters, using brute force attack, hacker have to take years to crack the password.

““

1.7 Task 4

If a given hash value is from a bcrypt algorithm, is it practical to do a brute-force attack?

Ans. No, according to task 3, using bcrypt, if we hack by brute force attack, it will take years to crack the password.

1.8 Task 5

If a given hash value is from a bcrypt algorithm, is it practical to perform a rainbow table attack?

Ans. No, it is not practical to perform a rainbow table attack on a bcrypt hash. Bcrypt incorporates a salt into the hashing process, which means that even if two users have the same password, their hashes will be different. This makes precomputed rainbow tables ineffective, as they would not account for the unique salts used in each hash.

1.9 Task 6

You have to store a password in a database. Please explain your design/strategy for securely storing it. (Hints: Proper hash function, Salting, Cost factor, Database Security)

Ans. Using a proper hash function like bcrypt, I would first generate a unique salt for each password before hashing it. The cost factor would be set to a level that balances security and performance, ensuring that the hashing process is slow enough to deter brute-force attacks. Additionally, I would ensure that the database is secured with proper access controls, encryption at rest, and regular security audits to protect against unauthorized access.
